

LAB 2 – Advanced REST API & Security

Technical Requirements & Design Standards (PRN232)

Assignment Requirement

Continue developing the ASP.NET Core RESTful API from Lab 1 for the Learning Management System (LMS).

Students must reuse and improve the existing Lab 1 project while maintaining:

- 3-layer architecture
- RESTful API design
- Docker deployment
- Swagger/OpenAPI integration
- Consistent response format
- Search / Sort / Paging / Field Selection / Expansion
- Coding conventions and clean architecture

This lab focuses on building a more production-ready API by integrating:

- Content Negotiation
 - Data Binding / Model Binding
 - Data Validation
 - Advanced Routing
 - Middleware
 - Authentication & Authorization
 - JWT Security
 - API Versioning
-

1. Architecture & Project Structure

Continue using the 3-layer architecture from Lab 1:

- API Layer (Controllers)

- Service Layer (Business Logic)
- Repository Layer (Data Access)

Requirements

- Controllers must not contain business logic.
 - Repositories must not contain business logic.
 - Business rules must be implemented inside the Service Layer.
 - Dependency Injection must be used correctly.
 - Existing clean architecture and coding conventions must be maintained.
-

2. Database & Domain Requirements

Reuse all tables and APIs from Lab 1.

Students must additionally implement authentication-related tables.

Required Table

```
User(  
    UserId int,  
    Username varchar(50),  
    PasswordHash varchar(255),  
    Role varchar(20)  
)
```

Optional Additional Tables

- RefreshToken
 - Permission
 - AuditLog
-

3. Model Types

Continue using the model separation from Lab 1:

- Entity Model
- Business Model
- Request Model
- Response Model

Additional Requirements

- Validation attributes must be implemented in Request Models.
- Entity Models must not be returned directly to clients.

4. Content Negotiation

The API must support multiple response formats.

Required Formats

- application/json
- application/xml

Requirements

- Configure XML formatter.
- API must return data based on the Accept header.
- Unsupported formats should return HTTP 406.

Example

Accept: application/json

Accept: application/xml

5. Data Binding / Model Binding

Students must correctly implement ASP.NET Core model binding.

Required Binding Types

Route Binding

[HttpGet("{id:int}")]

```
public IActionResult GetStudent([FromRoute] int id)
```

Query Binding

```
public IActionResult GetStudents(  
    [FromQuery] StudentQueryRequest request)
```

Body Binding

```
public IActionResult CreateStudent(  
    [FromBody] CreateStudentRequest request)
```

Header Binding

```
[FromHeader(Name = "X-Request-Id")]
```

6. Data Validation

All client input must be validated.

Required Features

Validation Attributes

Students must use:

- [Required]
- [StringLength]
- [Range]
- [EmailAddress]
- [Phone]
- [RegularExpression]

Example

```
public class CreateStudentRequest  
{  
    [Required]  
    [StringLength(100)]  
    public string FullName { get; set; }  
}
```

```
[Required]
[EmailAddress]
public string Email { get; set; }
}
```

FluentValidation

Students must implement FluentValidation for at least ONE Request Model.

Custom Validation

Students must implement at least ONE custom validation rule. Ex: FPTU Style: SE19886, CE18793..

7. Advanced Routing & API Versioning

Students must implement advanced routing features.

Required Features

Attribute Routing

```
[Route("api/students")]
```

Route Constraints

```
[HttpGet("{id:int}")]
```

Nested Resources

```
/api/courses/{courseId}/students
```

Named Routes

```
Name = "GetStudentById"
```

API Versioning (Required)

Students must implement API Versioning.

Example

/api/v1/students

/api/v2/students

8. Middleware

Students must implement custom middleware components.

Required Middleware

Global Exception Handling Middleware

Requirements:

- Handle unhandled exceptions globally
- Return consistent error responses
- Avoid exposing internal server details

Example Response

```
{  
  "success": false,  
  "message": "Internal server error",  
  "errors": null  
}
```

Logging Middleware

Students must implement request logging middleware.

The middleware should log:

- Request path
- HTTP method

- Execution time
- Response status code

9. Authentication, Authorization & JWT Security

Students must implement JWT-based authentication and authorization.

Authentication API

Required endpoint:

POST /api/auth/login

Request:

```
{
  "username": "admin",
  "password": "123456"
}
```

Response:

```
{
  "success": true,
  "data": {
    "accessToken": "...",
    "refreshToken": "...",
    "expiresIn": 3600
  }
}
```

JWT Requirements

Students must:

- Generate JWT tokens
- Validate JWT tokens
- Configure JWT authentication middleware

- Protect APIs using authorization attributes

Authorization

Protected APIs

[Authorize]

Role-based Authorization

[Authorize(Roles = "Admin")]

At least ONE admin-only endpoint is required.

Refresh Token

Students must implement Refresh Token flow.

Required endpoint:

POST /api/auth/refresh-token

Password Security

Passwords must NOT be stored as plain text.

Students must use:

- BCrypt
OR
 - ASP.NET Core PasswordHasher
-

10. Docker Deployment

Continue using Docker Desktop deployment from Lab 1.

Requirements

The project must include:

- Dockerfile
- docker-compose.yml

Both API and Database must run successfully using Docker Compose.

Additional Requirements

- JWT secret must be configured using environment variables.
 - API must connect to database container correctly.
-

11. Swagger / OpenAPI Documentation

Continue using Swagger/OpenAPI from Lab 1.

Swagger must additionally support:

- JWT authentication testing
- Authorized API testing

JWT Swagger Requirement

Students must configure Swagger Authorize button for JWT testing.